TECHNICAL CASE · AI ENGINEER ASSESSMENT

Noorderhaven claims agent

An agentic intake system that reads claim emails (Dutch, English, mixed) with their PDFs and photos, checks them against the policy API, looks for fraud signals, approves what is safe and hands everything else to a human, with the reasoning attached.

The LLM extracts, code decides.
One LLM call turns an email into a structured fact sheet. Every coverage check, payout, routing decision and outgoing email is deterministic, tested Python. Customer text is untrusted data: an injected instruction can at most produce a wrong *fact*, never an approval.

40 / 40
case claims processed end to end, mock API chaos on

0
failed writes, no duplicate records (idempotent retries)

91.7%
strict match on a blind holdout of 12 new claims (100% lenient)

0
safety violations in every eval (5 hard checks)

What happened to the 40 claims

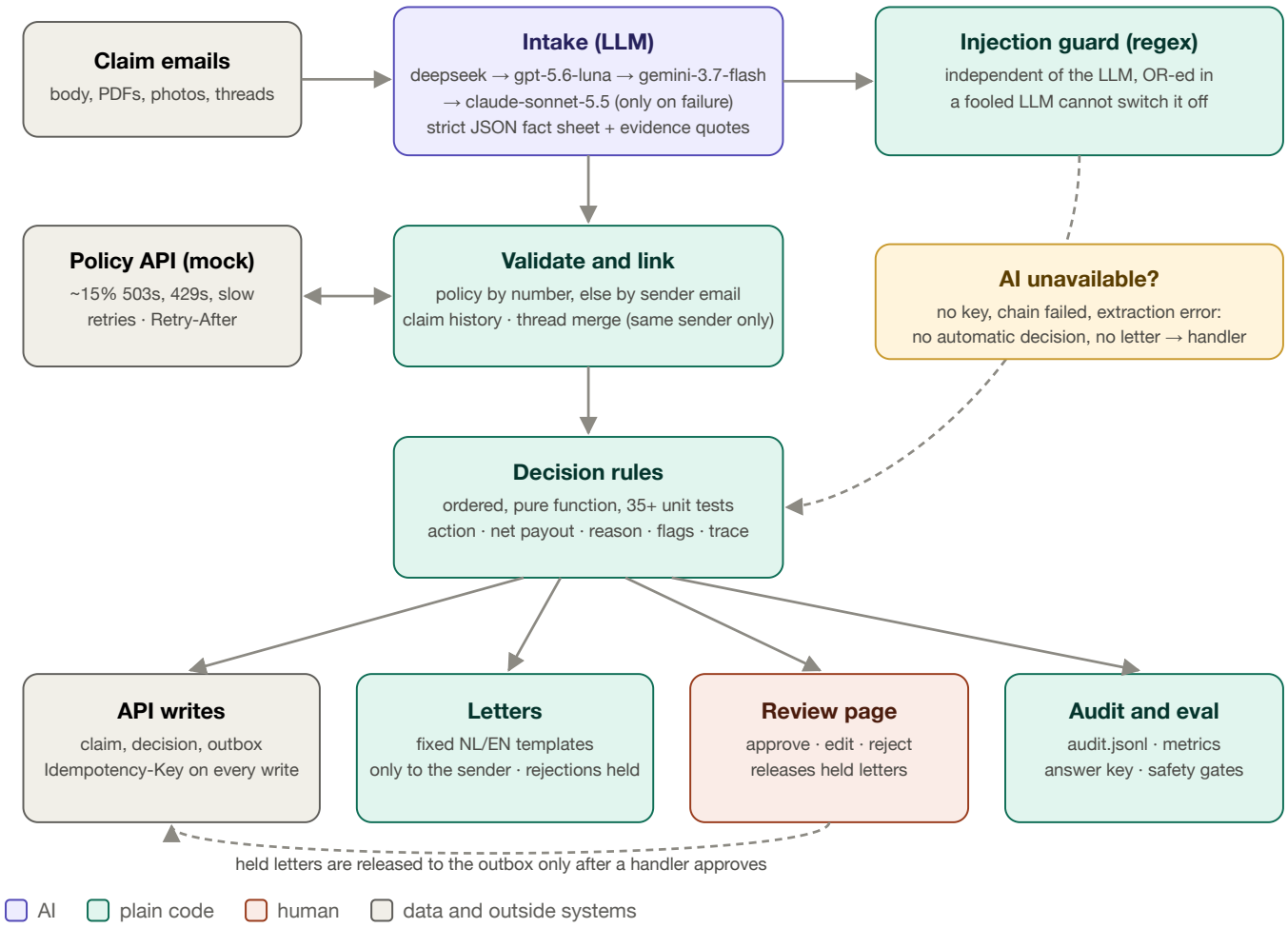
ACTION	CLAIMS	WHO ACTS NEXT
auto_approve	9	Approval letter sent; a handler samples these the next morning (§6)
request_info	4	Customer is asked for the missing document or identity (sent automatically, §6)
escalate_handler	8	Claims handler (above €1,000, or automation blocked: no photos, late report, policy found by email)
escalate_senior	3	Senior handler (net payout above €5,000)
reject_draft	7	Rejection letter held; a handler must approve it before it is sent
fraud_review	6	Fraud team (injection, duplicate invoice, repeat claimant, new policy, other bank account)
route_other	3	Spam, a complaint about an existing claim, a third-party liability claim

The seven parts

PART	WHAT IS BUILT	STRENGTH
01 Intake	LLM fact sheet with evidence quotes; stdlib PDF text; schema validated inside the retry loop; injection regex as a second, independent check	strong
02 Validation	Policy by number, else by sender email; active on the loss date; peril covered; missing info; identity (§5)	strong
03 Fraud signals	Duplicate invoice or event, 3+ claims in 12 months, loss within 30 days of start, other IBAN, instructions to the system	strong (listed indicators)
04 Decision	Net payout, auto / handler / senior bands, reason citing the rulebook, rule-by-rule trace	strong
05 Communication	NL/EN templates; info requests; held rejections; replies merged into their thread; open requests with reminder drafts	medium
06 Human review	Local web queue: facts, email, trace, draft letter; approve / edit / reject; every action logged	medium–strong
07 Monitoring	Per-step audit trail, run summary, metrics and alerts view, eval with safety gates, blind holdout, robustness runs	medium–strong

ARCHITECTURE

How one claim flows through the system



Design choices behind the diagram

- **One LLM call per email**, for extraction only. Each yes/no judgement (gradual damage, abroad, third party, other bank account, injection) comes with a verbatim evidence quote, so a handler can check it.
- **Model chain, cheapest first.** A schema failure counts as a failed attempt: retry, then the next model. The strongest model (Sonnet, about 25× the cost) is only used when the cheap ones fail.
- **Standard library only.** Nothing to install, no supply-chain surface, and it runs anywhere with Python 3.9+.
- **Sequential decisions, parallel I/O.** Extraction and policy lookups run in parallel. Decisions run in received order, so duplicate and repeat-claimant checks see claims registered earlier in the same run.
- **Retries are safe.** Every write carries a deterministic Idempotency-Key, so a retry after a timeout returns the original record instead of creating a second one.
- **The audit trail is the record.** Every step per claim is logged with its API attempt count. The mock API's own write log confirms what was actually written.

DECISION LOGIC

The rules run in a fixed order. The first one that applies decides.

#	CHECK	IF IT APPLIES
0	Was the email read by the LLM?	No (no key, chain down, extraction error) → **escalate_handler**, no letter, facts marked unverified
1	Intent: spam, complaint, third-party liability claim	**route_other** with a neutral acknowledgement that discloses nothing (§5)
2	Policy found by number or sender email?	No → **request_info** for identification (fraud_review if it also shows risk indicators)
3	Sender is the policyholder (§5)?	No → **request_info** for identification; nothing about the policy is disclosed
4	Risk indicators (§4): injection, other IBAN, duplicate invoice or event, 3+ claims in 12 months, loss within 30 days of start	**fraud_review** with every indicator listed
4b	Clear exclusion: peril not covered, gradual damage, outside Europe, accidental damage on contents	**reject_draft**, held for a handler (no point asking for an invoice first)
5	Essentials: loss date, what happened, priced document, police report for theft	**request_info** listing exactly what is missing
6	Policy active on the loss date; peril covered	**reject_draft**, held for a handler
7	Net payout = min(document amount, max payout) – deductible (glass deductible for glass)	The document beats the email amount (§2.6); a mismatch is flagged, not penalised
8	Soft blocks: reported over 30 days late, no photos above €500, policy found only via email	Automatic approval blocked → **escalate_handler**
9	Limits (§3)	≤ €1,000 and clean → **auto_approve**; ≤ €5,000 → **escalate_handler**; above → **escalate_senior**

SAFETY

Where a human stays in control

RISK	HOW IT IS LIMITED
Wrong automatic payout	Only net ≤ €1,000 with complete documents, a verified sender, the policy found by number, and no risk indicator, late report or missing photos. Handlers sample these. Eval gate: 0 false auto-approvals.
Prompt injection	The LLM decides nothing. Regex and LLM detection both send the claim to fraud review, and the review page shows the injected passage with a warning. Both case injections (in an email and hidden in a PDF) and a reworded Dutch one in the holdout were caught.
Data leak	Mail only goes to the original sender (checked in the pipeline and again in the review server). Identity requests carry no policy data. Third parties get nothing. Thread links across different senders are refused.
Rejection without a human	Rejections are always drafts until a handler approves them on the review page. Eval gate: 0.
AI outage	The keyword fallback never decides in normal mode. Every claim goes to a handler, with no letter sent.
Duplicates from retries	Deterministic Idempotency-Keys. The write log shows each claim and decision exactly once.
Review page abused from another site	JSON-only POSTs, plus Origin and Host checks (CSRF and DNS rebinding).

EVALUATION

How we know the decisions are right, and where that claim is weaker

Case set, 40 claims

100% action match and 100% payout match against an answer key written from the rulebook *before* the code. All 5 safety checks at 0.

Read that number carefully. Two rules were adjusted after the first eval on these same claims. The first untuned run scored 97.5%.

Blind holdout, 12 new claims

Written by an agent that never saw the code, with answers from the rulebook alone. **91.7% strict, 100% counting accepted alternatives**, payouts 5/5, safety 0.

The one strict miss: no photos above €500 went to a handler, where the author preferred asking the customer for photos.

Independent review

A separate reviewer session rebuilt an answer key without seeing the code: 37/40 same action. The 3 differences are rulebook gaps (the photos rule, a mistyped policy number). It also found the fallback safety gap, now fixed and tested.

Robustness

Chaos on in every run: 170 API calls, about 20% retried, 0 failed writes. Also tested: KIE down, an idempotent re-run, and no-LLM mode (see `docs/robustness.md`).

RULEBOOK GAPS

How unclear rules were read

GAP	READING
Email and document amounts differ	Not a §4 contradiction: §2.6 says the document counts. Flagged; automatic approval still allowed.
"Photos are expected" above €500	Blocks automatic approval, never rejects. Not applied to theft.
Reported late ("in principle 30 days")	More than 30 days goes to a handler, never an automatic rejection.
"Sudden"	Damage developing over weeks or months (damp, mould, slow leak) is gradual: rejection drafted, held.
Wrong or missing policy number	Look it up by sender email. If exactly one policy matches, the sender is verified, but the claim is never approved automatically.
Telling escalated customers anything?	Yes, a neutral acknowledgement: no amount, no promise, no fraud wording.
Reply from a different sender	Not linked to the thread (In-Reply-To is easily forged); processed on its own.

HONEST STATUS

What is cut, fake or untested

Cut on purpose

- **Vision on photos.** The photos are captioned placeholders, so we only count them.
- **A real mailbox and multi-turn reminders.** Replies are merged and reminder drafts listed, but nothing polls an inbox.
- **LLM-written letters.** Templates cannot promise money or leak data.
- **Logins, multiple handlers, a database.** State is JSON files; the mock API is the system of record.

Fake or untested

- No email is really sent and no money moves (mock outbox).
- The review page has no automated browser test; it was checked by hand and through its API.
- The final run replayed LLM answers cached from the live run minutes earlier (same prompt and models, 40 real calls, 1.19 credits).
- Explicit `--no-llm` mode is a developer mode. On unseen emails the keyword reader misses things, which is why it never decides in normal mode.

What breaks first at 200 claims a day

1. **LLM rate limits and latency.** KIE allows 20 requests per 10 s, and upstream errors happen. Fine for 200 a day, but a 09:00 burst queues. Next: a persistent work queue with per-provider rate limits.
2. **The flaky policy API.** Retries cope, but a crash mid-run needs a durable outbox to resume. Idempotency keys already make a restart safe.
3. **Handler capacity.** About 65% of claims need a human here. The value comes from tuning what is safe to automate, driven by the review log's override rate.
4. **JSON files as state.** Concurrent handlers would clash. Move to a database.

Next steps, in order

1. A persistent per-claim ledger checked before deciding, so a re-run never auto-approves a claim an earlier run left for a handler.
2. A reminder flow for info requests, with real inbox threading.
3. A persistent queue, a database, and multi-user review with logins.
4. A larger labelled eval set from handled claims, with the safety checks as a CI gate.
5. An LLM second opinion on thin "gradual" and "outside Europe" evidence, shown to the handler, never deciding.
6. A vision check on real photos.

Run it: `python3 mock_api/server.py`, then `python3 -m agent.run --reset`, then `python3 eval/run_eval.py`, then `python3 review/server.py` (review page on :8000). Standard library only. See README.md.